

Sensor Integration System

Design Document

Version: 1.0

Date: February 09, 2026

Team: Team #2

Table of Contents

[1. Purpose](#)

[2. Scope](#)

[2.1 Primary Functionality](#)

[2.2 User Interfaces](#)

[3. Exclusions, Assumptions, and Limitations](#)

[3.1 Exclusions](#)

[3.2 Assumptions](#)

[3.3 Limitations](#)

[4. Solution Design Overview](#)

[4.1 Problem Statement](#)

[4.2 Solution Architecture](#)

[4.3 Architecture Flow](#)

[5. Technical Architecture](#)

[5.1 Server Specifications](#)

[5.2 Technology Stack](#)

[5.3 Network & Integration](#)

[5.4 Infrastructure/Application Diagram](#)

[5.5 Physical Layout](#)

[6. Configuration Specification](#)

[6.1 Hardware and Software Components](#)

[6.2 Software Dependencies](#)

[6.3 Environment Variables](#)

[6.4 Database Configuration](#)

[7. Solution Design Specification](#)

[7.1 Software Description](#)

[7.2 Coding Standards](#)

[7.3 Data Design](#)

[7.4 Module Description](#)

- [8. Roles and Responsibilities](#)
- [9. Terms and Definitions](#)
- [10. Project Schedule](#)
- [11. Supporting References](#)
- [12. Appendix](#)
 - [12.1 API Endpoints](#)
 - [12.2 Troubleshooting](#)
- [13. Revision History](#)

1. Purpose

This document defines the technical architecture, configuration, and design specifications of the Sensor Integration System, a web platform for monitoring and managing IoT sensor devices. It serves as the primary technical reference for development, deployment, and maintenance.

Project Context: Educational project demonstrating full-stack development, containerized deployment, and cloud-native architecture.

Business Purpose: Provide centralized monitoring, configuration, and administration of distributed environmental sensors (temperature, humidity, pressure).

Related Documentation:

- Requirements: [README.md](#)
- Team workflow: [TEAM_WORKFLOW.md](#)
- Setup instructions: [SETUP_INSTRUCTIONS.md](#)
- Repository: <https://github.com/guytru28-art/sensor-integration-system>

2. Scope

The system supports end-to-end IoT device management, including user authentication, device onboarding, real-time monitoring, historical data access, and administrative oversight.

2.1 Primary Functionality

User Management:

- JWT-based registration and authentication
- Role-based access control (user/admin/super admin)
- Configurable auto-logout after inactivity (1-120 minutes)

Device Management:

- Device registration and categorization by type
- Online/offline status tracking
- Per-user device isolation with cascading data deletion

Sensor Data:

- Periodic data ingestion via REST API
- Real-time dashboard updates (~10s polling)
- Time-series storage (temperature, humidity, pressure)
- Historical data queries

Administration:

- System dashboards (users, devices, readings)
- User/device lifecycle management
- Privilege elevation (user → admin → super admin)
- Super admin controls: disable accounts, reset passwords, activity audit logs

2.2 User Interfaces

Web Interface:

- React SPA with responsive layout (mobile to desktop)
- Dashboard cards, modals, status indicators

Administrative Interface:

- Admin-only dashboards with statistical summaries
- Super admin panel with enhanced user controls
- Bulk user/device operations

API Interface:

- REST API with JSON payloads
- Bearer token authentication
- Device/system integration support

Supported Browsers: Chrome ≥90, Firefox ≥88, Safari ≥14, Edge ≥90

3. Exclusions, Assumptions, and Limitations

3.1 Exclusions

Not included in this release:

Communication:

- Email, SMS, push notifications
- 2FA, OAuth, CAPTCHA

Data Management:

- Automated backups, data export (CSV/Excel)
- Advanced analytics, ML prediction, mapping

Integration:

- External IoT platforms (AWS IoT, Google Cloud IoT)
- Cloud storage (S3, Azure Blob)
- Monitoring services (Datadog, New Relic, Google Analytics)

Other:

- Multi-language support, rate limiting
- WebSockets, MQTT, CoAP protocols

3.2 Assumptions

Technical:

- Modern browser with JavaScript enabled
- Stable internet (~1 Mbps minimum)
- Docker environment available for local development

Operational:

- ≤ 100 concurrent users
- ~5-10 devices per user
- Sensor reporting interval: 10-60 seconds

User:

- Basic web and IoT familiarity
- Secure user devices

3.3 Limitations

Performance:

- SQLite limits concurrency scaling

- Single-server deployment (no load balancing)
- Recommended $\leq 10k$ readings per device

Security:

- No rate limiting or CAPTCHA
- JWT tokens non-revocable server-side
- No automated backup or encryption at rest

Integration:

- Manual device setup only
- HTTP/HTTPS only (no MQTT/CoAP)
- No firmware OTA updates

Infrastructure:

- Hosting tier limits runtime (500 hrs/month)
- Container restarts clear in-memory state

4. Solution Design Overview

4.1 Problem Statement

IoT deployments commonly suffer from:

- Fragmented monitoring interfaces
- Isolated sensor data
- Limited access control
- Manual device management
- Lack of centralized visibility

4.2 Solution Architecture

The system uses a **three-tier architecture**:

Presentation Tier:

- React SPA with browser-based UI
- Periodic polling for real-time updates

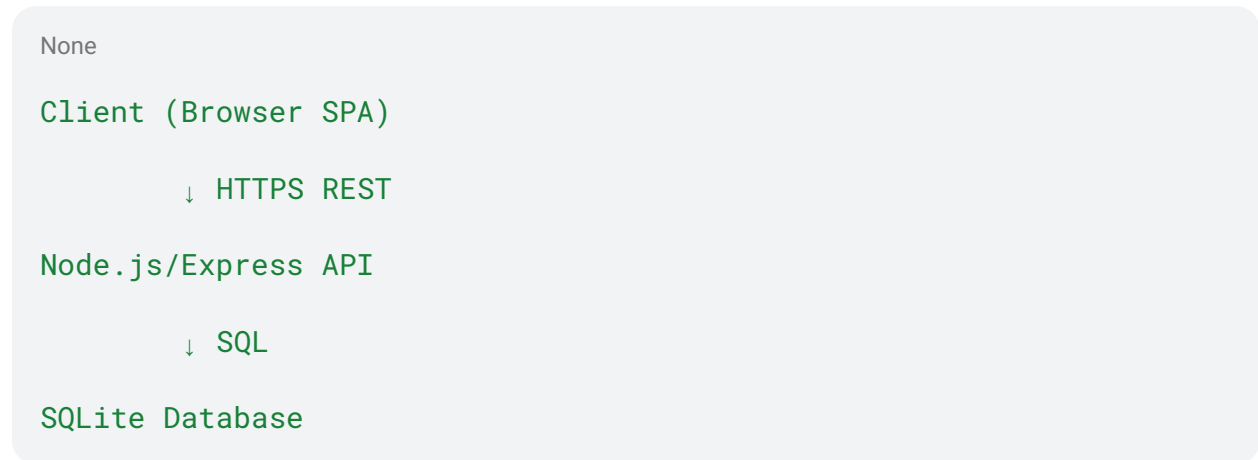
Application Tier:

- Node.js + Express.js REST API
- Authentication, authorization, device/data services

Data Tier:

- SQLite relational database
- Indexed time-series sensor data

4.3 Architecture Flow



This architecture provides:

- Centralized sensor monitoring
- Secure multi-user access
- Lightweight deployment for small-scale IoT environments

5. Technical Architecture

5.1 Server Specifications

Development Environment (Local Workstations):

- **Container:** node:18-alpine (~180MB)
- **Resources:** 512MB-1GB RAM, port 3001 exposed
- **Host OS:** Windows 10/11 (WSL2), macOS 11+, Linux (Ubuntu 20.04+)
- **Requirements:** 4-core CPU, 8GB RAM, 10GB free disk
- **Quantity:** 5 instances (one per team member)

Production Environment (Railway Cloud):

- **Provider:** Railway (<https://railway.app>)
- **Region:** us-west-1 (US West Coast)
- **Compute:** 1 shared vCPU, 512MB RAM, 1GB SSD
- **Service Tier:** Hobby Plan (500 hrs/month, auto-sleep after 30 mins)
- **URL:** <https://sensor-integration-system-production.up.railway.app>

Input/Output Devices:

- **User Input:** Standard keyboard (QWERTY), mouse/trackpad, capacitive touchscreens
- **Sensor Input (External):** DHT22 (temperature/humidity), BMP280 (barometric pressure), ESP8266/ESP32/Arduino microcontrollers
- **User Output:** Visual displays ranging from 375px (mobile) to 4K desktop monitors

Network and Development Tools:

- **Network:** 2.4 GHz WiFi for IoT devices, outbound HTTPS (port 443)
- **Software:** Git 2.30+, VS Code (recommended), Docker Desktop 20.10+, modern web browsers

5.2 Technology Stack

Backend:

- Node.js 18.x LTS
- Express.js 4.18.2
- SQLite 3.x
- JWT (jsonwebtoken 9.0.2)
- bcrypt (bcryptjs 2.4.3)

Frontend:

- React 18.2.0 (CDN: unpkg.com)
- Babel Standalone 7.22.5
- Custom CSS (responsive design)

DevOps:

- Docker containers
- Railway CI/CD
- Git/GitHub version control

Security:

- CORS (restricted to production domain)
- 1MB JSON payload limit
- JWT bearer token validation
- Role-based access control (RBAC)

5.3 Network & Integration

Production Network Flow:

1. DNS resolves to CloudFlare Anycast IP
2. Railway Load Balancer terminates SSL/TLS
3. Request forwarded to container (port 3001)
4. Express.js processes logic, queries SQLite
5. Encrypted response returned to user

Deployment Flow (CI/CD):

1. `git push origin main` triggers deployment
2. Railway pulls code, builds Docker image
3. New container spawns, health checks pass
4. Traffic switches (zero-downtime deployment)
5. Old container removed after 30 seconds

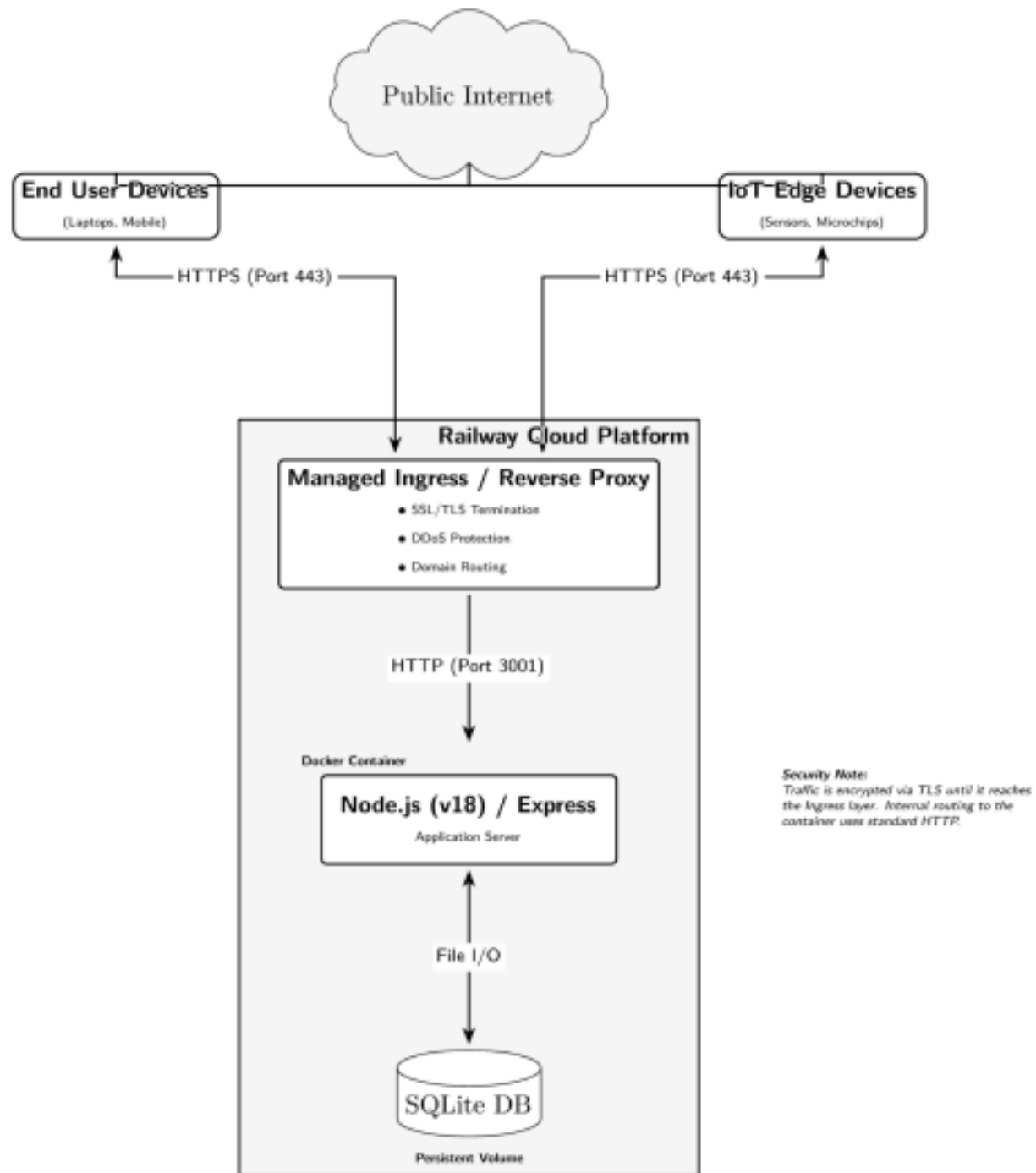
IoT Integration:

- **Endpoint:** `POST /api/sensor-data/:deviceId`
- **Format:** JSON (temperature, humidity, pressure)
- **Auth:** JWT in Authorization header
- **Unsupported:** MQTT, CoAP, WebSockets, third-party IoT hubs

External Dependencies:

- **CDNs:** React, ReactDOM, Babel via unpkg.com
- **Railway Services:** SSL (Let's Encrypt), DNS, logging (7-day retention)

5.4 Infrastructure/Application Diagram



5.5 Physical Layout

Development Environment:

- Developer workstations run Docker containers locally
- Source code stored on host filesystem, mounted into container via bind mount

- Port 3001 exposed from container to host machine (localhost:3001)
- SQLite database file persists on host at `./sensor_system.db`
- `node_modules` managed in separate Docker volume for performance

Production Environment:

- Railway cloud infrastructure (us-west-1 datacenter)
- Single Docker container deployment
- Application code and runtime packaged in container image
- Persistent SSD volume (1GB) mounted at `/app` for database storage
- Load balancer handles SSL/TLS termination and routes traffic to container
- Container accessible only via internal port 3001, external traffic via HTTPS (443)

Data Flow:

- **Development:** Browser → localhost:3001 → Docker container → SQLite file on host
- **Production:** Browser → HTTPS (443) → Railway LB → Container (3001) → SQLite on persistent volume

6. Configuration Specification

6.1 Hardware and Software Components

Development Environment (Local):

- **Container:** node:18-alpine, 512MB-1GB RAM
- **Networking:** Bridge mode (172.18.0.0/16), port 3001:3001
- **Volumes:** Bind mount (source code), named volume (`node_modules`)

Production Environment (Railway):

- **Compute:** 1 vCPU, 512MB RAM, 1GB persistent SSD
- **Network:** Dynamic IP, automatic HTTPS (port 443)
- **Runtime:** 500 monthly hours, 5-10s cold start

6.2 Software Dependencies

Backend (Node.js):

- `express` ^4.18.2 - routing, middleware, JSON parsing
- `bcryptjs` ^2.4.3 - password hashing (10 salt rounds)
- `jsonwebtoken` ^9.0.2 - JWT auth (HS256)
- `sqlite3` ^5.1.6 - async database interface
- `cors` ^2.8.5 - cross-origin requests
- `dotenv` ^16.3.1 - environment variables

Frontend (CDN):

- React/ReactDOM 18.2.0
- Babel Standalone 7.22.5

6.3 Environment Variables

Variable	Value	Notes
PORT	3001	Managed by Railway in production
JWT_SECRET	64+ char string	Critical: Never commit to Git, use random hex
NODE_ENV	development / production	Controls logging and optimizations

Security Practice: Production secrets injected via Railway dashboard. Local `.env` excluded via `.gitignore`.

6.4 Database Configuration

Engine: SQLite 3.x

Optimizations:

- **Journal Mode (WAL):** Write-Ahead Logging for concurrent reads during writes
- **Synchronous (NORMAL):** Balances speed with safety against power loss
- **Foreign Keys:** Enabled via `PRAGMA foreign_keys = ON`

Storage:

- Production: 1GB persistent SSD volume
- Integrity: `ON DELETE CASCADE` for devices and sensor_data
- Backup: Manual via Railway CLI/Dashboard

Schema (v1.0):

- **users:** Hashed credentials, admin/super admin flags, activity status
- **devices:** Hardware IDs (e.g., SENSOR001) linked to owners
- **sensor_data:** Time-series with composite indexes (device_id, timestamp)

- **activity_logs:** Audit trail (persists via SET NULL on user deletion)

7. Solution Design Specification

7.1 Software Description

System Type: Containerized, three-tier web application

Architecture: Event-driven, asynchronous I/O

Backend: Node.js 18 LTS + Express.js + SQLite 3

Frontend: React 18.2 (client-side rendering) + Babel (runtime JSX)

Security: bcrypt (Blowfish hashing) + JWT (HMAC SHA-256)

7.2 Coding Standards

Naming Conventions:

- Variables/functions: `camelCase`
- React components: `PascalCase`
- Constants: `SCREAMING_SNAKE_CASE`

Security:

- Parameterized SQL queries (prevent injection)
- 2-space indentation, required semicolons

Git Workflow:

- Commit format: `<type>: <description>`
- Example: `feat: add temperature alert threshold`

7.3 Data Design

Relational Structure: Users → Devices → Sensor Data

Schema Highlights:

- **users:** id, username, email, password (hashed), is_admin, is_super_admin, is_active, last_login, disabled_at, disabled_by
- **devices:** id, user_id (FK), device_name, device_type, device_id (unique), status, created_at
- **sensor_data:** id, device_id (FK), temperature, humidity, pressure, timestamp
- **activity_logs:** id, user_id, action, target_user_id, details, ip_address, timestamp

Indexes: Primary keys, foreign keys, composite (device_id + timestamp)

Data Validation:

- Temperature: -273.15°C to 1000°C
- Humidity: 0% to 100%
- Pressure: 300 hPa to 1100 hPa
- Email format validation
- Password minimum 6 characters

7.4 Module Description

Backend:

- `server.js` - Express app entry point, middleware stack, REST API routes, authentication/authorization logic
- `database.js` - SQLite lifecycle management, schema initialization, WAL mode configuration

Frontend (React):

- `App` - Top-level authentication state management, routing logic
- `Dashboard` - Main user workspace displaying device grid
- `DeviceCard` - Individual device display component with 10-second polling for sensor data
- `AdminPanel` - System-wide management interface for admins and super admins
- `UserProfileModal` - User profile settings including inactivity timeout configuration
- `AddDeviceModal` - Form for registering new sensor devices
- `CountdownTimer` - Visual countdown display for auto-logout timer
- `useInactivityTimer` - Custom React hook monitoring user interaction for auto-logout (1-120 minutes)

8. Roles and Responsibilities

Role	Team Member(s)	Primary Responsibilities
Project Lead	All	Technical direction, Railway deployment, code review, documentation

Frontend Dev	All	React components, UI/UX design, responsive CSS styling
Backend Dev	All	API endpoints, SQL schema design, authentication logic
DevOps	All	Docker configuration, Git workflow, CI/CD pipeline management
QA / Writer	All	Functional testing, bug tracking, documentation maintenance

System Administrator Responsibilities:

- Validate all system components are installed and configured correctly
- Ensure infrastructure (servers, workstations, devices) supports the solution
- Monitor system health and performance
- Manage user accounts and access permissions

User Responsibilities:

- Register and manage personal account
- Add, remove, and configure sensor devices
- Monitor sensor data via dashboard
- Report issues to system administrator

Super Admin Responsibilities:

- All admin capabilities plus enhanced controls
- Disable/enable user accounts
- Reset user passwords
- Promote/demote admin privileges
- Monitor activity audit logs
- Permanently delete users and data

9. Terms and Definitions

API: REST endpoints enabling frontend-backend communication.

Authentication: User identity verification via email/password and JWT tokens.

Authorization: Role-based access control (user/admin/super admin).

CORS: Browser security mechanism controlling cross-origin requests.

CRUD: Create, Read, Update, Delete - basic database operations.

Docker: Container platform ensuring consistent dev/prod environments.

IoT: Internet of Things - network of connected sensor devices.

JWT: JSON Web Token for stateless authentication (24-hour expiry).

SQLite: Embedded relational database with WAL mode for concurrency.

WAL: Write-Ahead Logging - allows simultaneous reads during writes.

10. Project Schedule

Environment Setup

- Dockerized Node.js environment configuration
- Railway project initialization
- SQLite schema creation

Core API & Auth

- User registration/login implementation
- Bcrypt password hashing
- JWT middleware and token generation

Device Management

- REST endpoints for CRUD operations on devices
- Initial React Dashboard layout
- Device status tracking

Data & UI Integration

- Sensor data ingestion API
- 10-second polling logic
- Real-time UI updates

Admin & Security

- Administrator Panel development

- Super admin role implementation
- Inactivity Timer integration
- Activity logging system
- Final QA testing and bug fixes

11. Supporting References

Official Documentation:

- Node.js Documentation: <https://nodejs.org/docs/latest-v18.x/api/>
- Express.js Documentation: <https://expressjs.com/en/4x/api.html>
- React Documentation: <https://react.dev/>
- SQLite Documentation: <https://www.sqlite.org/docs.html>
- Docker Documentation: <https://docs.docker.com/>
- Railway Documentation: <https://docs.railway.app/>

Package Documentation:

- bcryptjs: <https://www.npmjs.com/package/bcryptjs>
- jsonwebtoken: <https://www.npmjs.com/package/jsonwebtoken>
- cors: <https://www.npmjs.com/package/cors>

Project Resources:

- GitHub Repository: <https://github.com/quytru28-art/sensor-integration-system>
- Live Deployment: <https://sensor-integration-system-production.up.railway.app>
- Docker Hub (Base Image): https://hub.docker.com/_/node

Security Standards & Best Practices:

- OWASP Top 10 Web Application Security: <https://owasp.org/www-project-top-ten/>
- JWT Security Best Practices: <https://tools.ietf.org/html/rfc8725>
- REST API Design Guidelines: <https://restfulapi.net/>
- bcrypt Hashing Guide: <https://auth0.com/blog/hashing-in-action-understanding-bcrypt/>

Educational Resources:

- React Hooks Documentation: <https://react.dev/reference/react>
- Docker for Beginners: <https://docker-curriculum.com/>
- SQLite Tutorial: <https://www.sqlitetutorial.net/>

12. Appendix

12.1 API Endpoints

Authentication:

- **POST** `/api/auth/register` - Create new user account
 - Body: { username, email, password }
 - Returns: { token, user }
- **POST** `/api/auth/login` - Authenticate user and return JWT
 - Body: { email, password }
 - Returns: { token, user }
- **GET** `/api/auth/me` - Get current user information
 - Headers: Authorization: Bearer <token>
 - Returns: { user }

Devices:

- **GET** `/api/devices` - List all devices owned by current user
 - Headers: Authorization: Bearer <token>
 - Returns: [{ id, device_name, device_type, device_id, status, created_at }]
- **POST** `/api/devices` - Add new device
 - Headers: Authorization: Bearer <token>
 - Body: { device_name, device_type, device_id }
 - Returns: { device }
- **DELETE** `/api/devices/:id` - Delete device and associated sensor data
 - Headers: Authorization: Bearer <token>
 - Returns: { message }
- **PATCH** `/api/devices/:id/status` - Update device status
 - Headers: Authorization: Bearer <token>
 - Body: { status: "online" | "offline" }
 - Returns: { device }

Sensor Data:

- **GET** `/api/sensor-data/:deviceId` - Get sensor readings for device
 - Headers: Authorization: Bearer <token>
 - Query: ?limit=50 (optional)
 - Returns: [{ id, temperature, humidity, pressure, timestamp }]
- **POST** `/api/sensor-data/:deviceId` - Submit sensor data reading
 - Headers: Authorization: Bearer <token>
 - Body: { temperature, humidity, pressure }
 - Returns: { message, data }

- **POST** `/api/demo/generate-data/:deviceId` - Generate test sensor data
 - Headers: `Authorization: Bearer <token>`
 - Returns: `{ message, data }`

Admin (Requires Admin Role):

- **GET** `/api/admin/users` - List all users with device counts
 - Headers: `Authorization: Bearer <token>`
 - Returns: `[{ id, username, email, is_admin, device_count, created_at }]`
- **GET** `/api/admin/devices` - List all devices system-wide
 - Headers: `Authorization: Bearer <token>`
 - Returns: `[{ id, device_name, username, email, reading_count }]`
- **GET** `/api/admin/stats` - Get system statistics
 - Headers: `Authorization: Bearer <token>`
 - Returns: `{ totalUsers, totalDevices, totalReadings, newUsersToday }`
- **DELETE** `/api/admin/users/:id` - Delete user and all associated data
 - Headers: `Authorization: Bearer <token>`
 - Returns: `{ message }`
- **DELETE** `/api/admin/devices/:id` - Delete any device
 - Headers: `Authorization: Bearer <token>`
 - Returns: `{ message }`
- **PATCH** `/api/admin/users/:id/make-admin` - Promote user to admin
 - Headers: `Authorization: Bearer <token>`
 - Returns: `{ message }`

Super Admin (Requires Super Admin Role):

- **PATCH** `/api/superadmin/users/:id/toggle-active` - Disable/enable user account
 - Headers: `Authorization: Bearer <token>`
 - Returns: `{ message, is_active }`
- **POST** `/api/superadmin/users/:id/reset-password` - Reset user password
 - Headers: `Authorization: Bearer <token>`
 - Returns: `{ message, tempPassword, username, email }`
- **PATCH** `/api/superadmin/users/:id/make-admin` - Promote user to admin
 - Headers: `Authorization: Bearer <token>`
 - Returns: `{ message }`
- **PATCH** `/api/superadmin/users/:id/remove-admin` - Demote admin to user

- Headers: `Authorization: Bearer <token>`
 - Returns: `{ message }`
- `DELETE /api/superadmin/users/:id/permanent-delete` - Permanently delete user
 - Headers: `Authorization: Bearer <token>`
 - Returns: `{ message, deleted_user }`
- `GET /api/superadmin/activity-logs` - View activity audit logs
 - Headers: `Authorization: Bearer <token>`
 - Query: `?limit=100` (optional)
 - Returns: `[{ id, user_id, action, target_user_id, details, ip_address, timestamp }]`
- `GET /api/superadmin/users/:id/details` - Get detailed user information
 - Headers: `Authorization: Bearer <token>`
 - Returns: `{ id, username, email, is_admin, is_super_admin, is_active, device_count, reading_count, last_login }`

12.2 Troubleshooting

Port 3001 Already in Use:

- **Linux/Mac:** `fuser -k 3001/tcp` or `lsof -ti:3001 | xargs kill`
- **Windows:** Change PORT in `.env` file or use Task Manager to end process
- **Docker:** `docker-compose down` then `docker-compose up`

Database Locked Error:

- Close any external database browsers (e.g., DB Browser for SQLite)
- Ensure only one application instance is accessing the database
- Check for crashed processes holding database locks
- Solution: Restart server or delete `.db-shm` and `.db-wal` files

Docker Connection Issues:

- Verify Docker Desktop is running (check system tray/menu bar)
- **Windows:** Ensure WSL2 integration is enabled in Docker Desktop settings
- **Mac:** Check Docker Desktop has necessary permissions
- Restart Docker Desktop if containers won't start
- Clear Docker cache: `docker system prune -a`

Changes Not Appearing in Browser:

- Hard refresh: `Ctrl+Shift+R` (Windows/Linux) or `Cmd+Shift+R` (Mac)

- Clear browser cache: `Ctrl+Shift+Delete`
- Try incognito/private browsing mode
- Verify file was saved in code editor
- Check Docker logs for server restart: `docker-compose logs -f`

Database Not Creating Tables:

- Delete existing database file: `rm sensor_system.db`
- Restart server to recreate schema
- Check `database.js` for syntax errors in CREATE TABLE statements

JWT Token Errors (401 Unauthorized):

- Token may have expired (24-hour lifetime)
- Log out and log back in to get new token
- Check Authorization header format: `Bearer <token>`
- Verify JWT_SECRET matches between dev and production

Railway Deployment Fails:

- Check Railway dashboard logs for error messages
- Verify all environment variables are set correctly
- Ensure `package.json` scripts include `"start": "node server.js"`
- Check that port 3001 is specified in environment variables

Super Admin Setup Not Working:

- Verify you're using correct secret code
- Check browser console for error messages
- Ensure you're logged in before running setup command
- Try direct database access via Railway CLI if console method fails

13. Revision History

Version	Date	Author	Description of Changes
0.1	2026-02-15	Team #2	Initial draft - project structure and requirements

0.5	2026-02-18	Team #2	Added backend implementation (authentication, devices, sensor data)
0.8	2026-02-19	Team #2	Added admin panel functionality and user management
1.0	2026-02-19	Team #2	Initial release - complete documentation for basic features
1.1	2026-02-20	Team #2	Added super admin features, activity logging, physical layout section, supporting references

Document Status: Final Release (Version 1.1)

Next Review Date: March 1, 2026

Document Owner: Team #2

Approval Status: Approved for Class Submission

Change Summary (v1.1):

- Added Super Admin role and capabilities throughout document
- Enhanced Section 5.4 with explicit Physical Layout subsection
- Added Section 11 (Supporting References) with official documentation links
- Expanded API endpoint documentation in Appendix 12.1
- Added super admin API endpoints to appendix
- Updated troubleshooting guide with super admin setup instructions
- Enhanced Roles and Responsibilities section with super admin role